

PART XXI

EXCEPTIONS, FILES AND TEMPLATES

Error handling, stream I/O, and a full return to generic programming

This Part closes out the university-syllabus “additional mechanisms” started in Part XX: how C++ handles errors that can't be handled where they occur (exceptions), how a program reads and writes actual files instead of just the console (file streams), and a full, proper treatment of templates — picking up exactly where Part XX's brief introduction left off. None of this is needed for Codeforces or LeetCode (exceptions and file I/O essentially never appear in competitive judges), but templates in particular are worth understanding deeply: they're the mechanism the entire STL is built on.

Contents of Part XXI

- 1. Exception Handling Basics
- 2. Multiple Catch Blocks and Exception Hierarchies
- 3. Standard and Custom Exceptions
- 4. File I/O — Reading and Writing Files
- 5. Function Templates, Overloading, and Deduction
- 6. Template Specialization
- 7. Templates vs. Macros
- 8. Worked Example: Bubble Sort Using Templates
- 9. Worked Example: A Templated Linked List
- Part XXI Revision, Cheat Sheet, and Interview Questions

Chapter 1 — Exception Handling Basics

In one sentence: an exception is a way for code that detects an error to signal it (throw) to some other code, possibly much further up the call chain, that knows how to handle it (catch) — without every function in between needing to check for and pass along an error code manually.

```
double safeDivide(double a, double b) {  
    if (b == 0) {  
        throw runtime_error("Division by zero"); // THROW: signal the  
        error  
    }  
    return a / b;  
}  
  
int main() {  
    try {  
        cout << safeDivide(10, 0);  
    } catch (const runtime_error &e) { // CATCH: handle it  
        cout << "Error: " << e.what() << endl;  
    }  
    cout << "Program continues normally\n";  
}
```

Without the try/catch, an uncaught exception terminates the program (via a call to `std::terminate`) after printing a diagnostic message — exceptions are not optional to handle once thrown; they propagate up the call stack, unwinding each function's local objects (calling their destructors along the way) until either a matching catch block is found or the program ends.

1.1 Why exceptions instead of error-code return values?

	Error codes (C-style)	Exceptions
Propagation	Every intermediate function must check and re-return the code	Automatically propagates until a handler is found
Can be ignored silently?	Yes — a caller can just not check the return value	No — an uncaught exception terminates the program loudly

	Error codes (C-style)	Exceptions
Cost when NOT thrown	None	Essentially none (modern C++ implementations)
Cost when thrown	None (it's just a return)	Noticeably higher — not for routine control flow

♦ Practical rule

Reserve exceptions for genuinely exceptional, rare situations (a malformed file, an invalid argument that should never happen in correct usage) — never use them for routine, expected control flow (like “end of a normal search”), both because of their performance cost when thrown and because it makes ordinary logic harder to follow.

Chapter 2 — Multiple Catch Blocks and Exception Hierarchies

A single try block can be followed by multiple catch blocks, each handling a different exception type — checked in order, top to bottom, and only the first matching one runs.

```
try {
    riskyOperation();
} catch (const invalid_argument &e) {
    cout << "Bad argument: " << e.what() << endl;
} catch (const out_of_range &e) {
    cout << "Out of range: " << e.what() << endl;
} catch (const exception &e) { // catches ANY std::exception not
    // matched above
    cout << "Other error: " << e.what() << endl;
} catch (...) { // catches literally anything else
    cout << "Unknown error" << endl;
}
```

⚠ Common mistake

Ordering catch blocks from most general to most specific (e.g. catching exception before `invalid_argument`). Since `invalid_argument` IS-A exception, the general catch would match first and the specific one would never run — some compilers warn about this, but it's easy to miss. Always order catch blocks from MOST specific to LEAST specific.

This ordering requirement exists precisely because the standard exception types form an inheritance hierarchy (Chapter 5-style inheritance from Part XX) — a `catch(const exception&)` matches any derived exception type via the same base-pointer/reference polymorphism covered in that Part.

Chapter 3 — Standard and Custom Exceptions

3.1 The `std::exception` hierarchy

Exception type	Typical meaning
<code>std::exception</code>	Base class for all standard exceptions
<code>std::logic_error</code>	A problem detectable before runtime, in principle (base for the next two)
<code>std::invalid_argument</code>	An argument doesn't make sense for the function
<code>std::out_of_range</code>	An index or value is outside a valid range (e.g. <code>vector::at</code>)
<code>std::runtime_error</code>	A problem only detectable while running (base for the next one)
<code>std::overflow_error</code>	An arithmetic operation overflowed
<code>std::bad_alloc</code>	Memory allocation (new) failed

`vector::at(i)` throws `std::out_of_range` when `i` is invalid, unlike `operator[]` which has undefined behavior on an invalid index — a small but genuinely useful safety difference worth remembering from Part IV.

3.2 Defining a custom exception class

```
class InsufficientFundsException : public runtime_error {
public:
    InsufficientFundsException(double shortfall)
        : runtime_error("Insufficient funds, short by " +
            to_string(shortfall)) {}
};

class BankAccount {
    double balance;
public:
    BankAccount(double b) : balance(b) {}
    void withdraw(double amount) {
        if (amount > balance) {
            throw InsufficientFundsException(amount - balance);
        }
        balance -= amount;
    }
};
```

Deriving your custom exception from a standard exception type (here, `runtime_error`) means existing code that catches `const exception&` automatically handles your custom exception too, while more specific code can still catch `const InsufficientFundsException&` directly for special handling.

Practice — Chapter 3

Problem 21.1 — Validated Stack with Custom Exceptions [Level 2]

Hint 1: Popping or peeking an empty stack should be a clear, catchable error, not undefined behavior.

Approach: Define a `StackEmptyException` (deriving from `runtime_error`) and throw it from `pop()/top()` when the stack is empty.

Chapter 4 — File I/O: Reading and Writing Files

C++'s file streams (from `<fstream>`) work almost identically to `cin/cout` from Part I — the same `>>` and `<<` operators, just connected to a file instead of the console.

4.1 Writing to a file

```
#include <fstream>

void writeScores() {
    ofstream out("scores.txt"); // ofstream = output file stream
    if (!out) { // check the stream opened successfully
        cout << "Failed to open file for writing\n";
        return;
    }
    out << "Alice 95\n";
    out << "Bob 88\n";
    out.close(); // flushes and releases the file (also happens
                // automatically
                // when 'out' goes out of scope, via its destructor)
}
```

4.2 Reading from a file

```
void readScores() {
    ifstream in("scores.txt"); // ifstream = input file stream
    if (!in) {
        cout << "Failed to open file for reading\n";
        return;
    }
    string name;
    int score;
    while (in >> name >> score) { // reads until end-of-file or a
        bad read
        cout << name << " scored " << score << "\n";
    }
    in.close();
}

// reading a whole file line by line
void readLines() {
    ifstream in("scores.txt");
    string line;
    while (getline(in, line)) {
        cout << line << "\n";
    }
}
```

`while (in >> name >> score)` relies on the stream converting to a boolean success/failure state — the loop naturally stops the moment a read fails (typically end of file), which is the idiomatic way to consume a file of unknown length without tracking a separate count.

4.3 fstream for both reading and writing, and append mode

```
fstream file("data.txt", ios::in | ios::out); // read AND write
ofstream log("log.txt", ios::app); // append instead of overwrite
ofstream binOut("data.bin", ios::binary); // binary mode -- no
text translation
```


⚠ Common mistake

Forgetting to check whether a file stream opened successfully before using it (a missing file, a permissions error, or a bad path all fail silently by default). Always check `if (!stream)` or `stream.is_open()` immediately after opening, since a failed stream simply reads nothing / writes nothing rather than throwing by default.

Practice — Chapter 4**Problem 21.2 — Word Frequency Counter from a File [Level 2]**

Hint 1: Read the file word by word using the `>>` operator, which already splits on whitespace.

Approach: Open the file with `ifstream`, read words in a loop, tally counts in a map, then print the results.

Chapter 5 — Function Templates, Overloading, and Deduction

Part XX introduced the basic template shape. This chapter goes deeper: how the compiler decides which type `T` to use, and how templates interact with ordinary function overloading.

5.1 Template argument deduction

```
template <typename T>
void printTwice(T x) {
    cout << x << " " << x << endl;
}

printTwice(5); // T deduced as int, from the argument
printTwice(3.14); // T deduced as double
printTwice<float>(5); // T EXPLICITLY specified as float,
                     // overriding deduction
```

The compiler deduces `T` by matching the actual argument's type against the template parameter's position — usually straightforward, but ambiguous in some cases (e.g. mixed-type arguments to a

two-parameter-same-type template), which is exactly when explicit specification (`printTwice<float>(5)`) is needed.

5.2 Function-template overloading

A template can be overloaded just like an ordinary function — by a different number of parameters, or by mixing a template with a fully concrete (non-template) overload for a specific type.

```
template <typename T>
T addAll(T a, T b) { return a + b; }

template <typename T>
T addAll(T a, T b, T c) { return a + b + c; } // overload:
different parameter count

// a concrete (non-template) overload for a specific case
string addAll(const char *a, const char *b) {
    return string(a) + string(b);
}
```

Overload resolution rule of thumb: when both a non-template function and a template could match a call equally well, the compiler prefers the non-template (concrete) version — this is exactly what allows library authors to provide a generic template plus hand-tuned overloads for specific important types, without the generic version ever “winning” by accident.

5.3 Multiple template parameters

```
template <typename K, typename V>
class KeyValue {
    K key;
    V value;
public:
    KeyValue(K k, V v) : key(k), value(v) {}
    void print() const { cout << key << " -> " << value << endl; }
};
// KeyValue<string, int> kv("age", 25); -- exactly how
std::pair/std::map work internally
```

Practice — Chapter 5

Problem 21.3 — Generic Swap and Generic Min/Max [Level 1]

Hint 1: These should work identically for int, double, and any type supporting comparison/assignment.

Approach: `template void mySwap(T &a, T &b);` `template T myMin(T a, T b);`

Chapter 6 — Template Specialization

In one sentence: template specialization lets you provide a completely different implementation of a template for one specific type, while every other type still uses the generic version.

```
template <typename T>
class Container {
    T value;
public:
    Container(T v) : value(v) {}
    void describe() const { cout << "Generic container holding: " <<
value << endl; }
};

// full specialization for bool -- a completely different
implementation
template <>
class Container<bool> {
    bool value;
public:
    Container(bool v) : value(v) {}
    void describe() const {
        cout << "Boolean container holding: " << (value ? "true" : "
false") << endl;
    }
};
```

This is exactly why `std::vector<bool>` behaves differently from every other `vector<T>` — the standard library provides a full specialization for `bool` that packs 8 booleans into a single byte instead of storing each as a full byte, trading some interface quirks (no real `bool&` references

into it) for major space savings.

✦ When to reach for specialization

Use it when a generic algorithm's default behavior is wrong, unsafe, or inefficient for one particular type, and that type is common enough to justify a dedicated implementation. Overusing specialization for many types instead of writing a genuinely generic algorithm defeats the purpose of templates in the first place.

Chapter 7 — Templates vs. Macros

Before templates, generic-looking code was often written with preprocessor macros (`#define`). Understanding why templates replaced macros for this purpose is a common exam and interview question.

```
#define MAX_MACRO(a, b) ((a) > (b) ? (a) : (b)) // macro version

template <typename T>
T maxTemplate(T a, T b) { return a > b ? a : b; } // template
version

int x = 5;
cout << MAX_MACRO(x++, 10); // BUG: x++ is textually substituted
TWICE,
    // so x is incremented twice in one call
cout << maxTemplate(x++, 10); // safe: x++ is evaluated exactly
once, as a normal argument
```

	Macros (<code>#define</code>)	Templates
Type checking	None — pure text substitution before compilation	Full type checking by the compiler
Side-effect safety	Unsafe — arguments can be evaluated multiple times	Safe — arguments evaluated normally, once
Debuggable?	Hard — debugger sees only the expanded text	Yes — debugger sees real typed functions
Scope	None — macros are global text replacement	Normal C++ scoping rules apply

♦ Practical rule

Prefer templates (or, for simple constant-like values, `constexpr/const`) over macros for anything that needs to behave like generic code.

Macros remain useful in modern C++ mainly for include guards and conditional compilation (`#ifdef`), not for generic logic.

Chapter 8 — Worked Example: Bubble Sort Using Templates

A direct application tying together Part VI's sorting chapter and this Part's templates: one bubble sort implementation that works for any type supporting `<` and `swap`.

```
template <typename T>
void bubbleSort(vector<T> &a) {
    int n = a.size();
    for (int i = 0; i < n - 1; i++) {
        bool swapped = false;
        for (int j = 0; j < n - 1 - i; j++) {
            if (a[j + 1] < a[j]) { // works for int, double, string, or any
                swap(a[j], a[j + 1]); // type with operator< and swap defined
                swapped = true;
            }
        }
        if (!swapped) break; // already sorted -- classic bubble sort
        early exit
    }
}

// works unchanged for any of these:
vector<int> nums = {5, 2, 8, 1};
vector<string> words = {"banana", "apple", "cherry"};
bubbleSort(nums); // T deduced as int
bubbleSort(words); // T deduced as string
```

This is a direct illustration of why template code, not runtime polymorphism, is the STL's usual choice for algorithms: `bubbleSort` here has zero runtime overhead compared to writing a dedicated `int` version and a dedicated `string` version by hand — the compiler generates both from the single definition at compile time.

Practice — Chapter 8

Problem 21.4 — Generic Insertion Sort with a Custom Comparator [Level 2]

Hint 1: Take a comparator function/lambda as an extra template parameter, similar to how `sort()` accepts one.

Approach: template `void insertionSort(vector&, Compare cmp)`, using `cmp(a, b)` in place of `a < b`.

Chapter 9 — Worked Example: A Templated Linked List

A full, generic singly linked list — combining nested classes (Part XX), constructors/destructors, and templates into one worked design.

```
template <typename T>
class LinkedList {
    struct Node {
        T data;
        Node *next;
        Node(T val) : data(val), next(nullptr) {}
    };
    Node *head = nullptr;
    Node *tail = nullptr;
    int count = 0;

public:
    ~LinkedList() { // destructor: free every node
        Node *cur = head;
        while (cur) {
            Node *next = cur->next;
            delete cur;
            cur = next;
        }
    }

    void pushBack(T val) {
        Node *n = new Node(val);
        if (!head) head = tail = n;
        else { tail->next = n; tail = n; }
        count++;
    }

    T& at(int index) {
        if (index < 0 || index >= count) {
            throw out_of_range("LinkedList index out of range");
        }
        Node *cur = head;
        for (int i = 0; i < index; i++) cur = cur->next;
        return cur->data;
    }

    int size() const { return count; }
};

// LinkedList<int> intList; LinkedList<string> stringList;
// both generated from this single definition, at compile time
```

This single class brings together almost every mechanism from Parts XX-XXI: a nested class (Node), a destructor that correctly frees every dynamically allocated node (avoiding the memory leak that a missing destructor would cause), an exception (out_of_range) thrown from at() exactly like std::vector::at, and the whole class parameterized as a template so it works for any element type without rewriting anything.

 Where this connects back

This is the same singly-linked-list shape from Part VIII, generalized. Everything you already know about traversal, insertion, and the classic pointer-manipulation bugs from that Part (dangling pointers, forgetting to update tail) applies unchanged here — templates only change what TYPE of data each node holds, never the underlying pointer logic.

Practice — Chapter 9

Problem 21.5 — Add a Templated Stack Class [Level 2]

Hint 1: A stack can be implemented as a thin wrapper around a templated singly linked list, pushing/popping only at the head.

Approach: template class Stack using a private LinkedList or raw Node* head, with push/pop/top/empty operating only at the front.

Part XXI — Master Revision

Must Remember

- Exceptions propagate automatically up the call stack until a matching catch is found; an uncaught exception terminates the program.
- Reserve exceptions for genuinely exceptional situations, not routine control flow — they have real cost when thrown.
- Order catch blocks from MOST specific to LEAST specific — a general catch(const exception&) placed first would swallow every more specific handler below it.
- `vector::at()` throws `std::out_of_range` on an invalid index; `operator[]` does not check and is undefined behavior instead.
- Always check a file stream opened successfully before using it — a failed open fails silently, it doesn't throw by default.
- When both a template and a non-template overload match a call equally well, the compiler prefers the non-template version.
- Template specialization lets one specific type get a completely different implementation, while every other type uses the generic version (e.g. `std::vector`).
- Templates beat macros for generic code because templates type-check and evaluate arguments exactly once; macros are blind text substitution and can silently double-evaluate side effects.

Common Mistakes Recap

Mistake	Fix
Ordering catch blocks general-to-specific	Order MOST specific exception types first
Using exceptions for routine, expected control flow	Reserve exceptions for genuinely rare/erroneous situations
Not checking whether a file stream opened successfully	Check <code>if (!stream)</code> or <code>.is_open()</code> immediately after opening

Mistake	Fix
Using a macro for generic min/max-style logic	Use a function template instead — avoids double-evaluation bugs
Forgetting a destructor in a manually-managed templated data structure	Always free every dynamically allocated node in the destructor, regardless of the element type T

Interview-Style Questions

- 1 Why must catch blocks be ordered from most specific to least specific?
- 2 Why does `std::vector::at()` throw an exception while `operator[]` does not?
- 3 Why should exceptions be reserved for exceptional situations rather than routine control flow?
- 4 Why can a macro like `#define MAX(a,b) ((a)>(b)?(a):(b))` silently evaluate an argument with side effects more than once, while a template function cannot?
- 5 What is template specialization, and why does `std::vector` use it?
- 6 Why does a templated linked list's destructor still need to explicitly free every node, regardless of what type T is?

Cheat Sheet — Part XXI

```
// Exceptions
try { throw runtime_error("message"); }
catch (const specific_error &e) { /* most specific first */ }
catch (const exception &e) { cout << e.what(); }
catch (...) { /* anything else */ }

// File I/O
ofstream out("file.txt"); if (!out) { /* handle failure */ }
ifstream in("file.txt"); while (in >> x) { ... } or while
(getline(in, line))
ofstream log("file.txt", ios::app); // append mode

// Templates
template <typename T> T fn(T a, T b) { ... } // function template
template <typename T> class C { T v; ... }; // class template
template <> class C<bool> { ... }; // full specialization
// prefer templates over #define macros for anything generic
```

Coming up in Part XXII

Part XXII — Competitive Programming Patterns steps back into the book's main spine: a dedicated pattern-recognition guide (how to spot a sliding-window, DSU, or DP problem from its phrasing alone) and the full constraints-to-algorithm-selection framework, pulling together recognition habits that have been built up piece by piece since the Graphs and Greedy Parts.